



**Auckland University of Technology**

**ENGE707 – Data Engineering and AI**

**Assessment 1**

**Phase 1 – Progress Report**

**Group:**

**Sravani Kunapuli – 23198431**

**Jaskaran Singh - 23210369**

## Table of Contents

|                                                          |    |
|----------------------------------------------------------|----|
| Introduction .....                                       | 3  |
| Task 1 Problem Definition and Dataset Selection.....     | 3  |
| 1.1 Problem Definition and Motivation .....              | 3  |
| 1.2 Dataset Selection and Relevance .....                | 4  |
| Task 2. Data Acquisition and Inspection .....            | 5  |
| 2.1 Dataset Structure and Initial Inspection .....       | 5  |
| 2.2 Initial Data Quality Findings .....                  | 6  |
| Task 3. Data Cleansing and Transformation .....          | 7  |
| 3.1 Missing Values and Data Cleaning.....                | 7  |
| 3.2 Data Transformation and Encoding .....               | 8  |
| 3.3 Outlier Investigation and Final Validation .....     | 10 |
| Task 4. Exploratory Data Analysis and Visualisation..... | 12 |
| 4.1 Descriptive Analysis and Data Distributions.....     | 12 |
| 4.2 Target Distribution and Variable Relationships ..... | 14 |
| 4.3 Correlation and Key EDA Findings.....                | 16 |
| 5. Data Pipeline and Reproducibility .....               | 18 |
| 6. Progress Summary .....                                | 19 |

# Introduction

This project is about rainfall prediction using the weather data that was collected in Australia. The real-world problem that this dataset addresses is the prediction of rainfall in Australia. The dataset chosen was the WeatherAUS (Graham Williams, 2010) dataset which contains historical Australian weather observation. Using this data, we can build a model that is able to predict if it will rain or not the following day. In real-world datasets often contains a lot of information that can be missing, such as invalid values, information that is not recorded, or even irrelevant data. Some values can also be categorised as inconsistent or unusual, therefore the data must be inspected and prepared before modelling. If these issues are not addressed the predictions of the future model can be severely inaccurate and of no use. Phase 1 of this project focus on acquisition, inspection, cleaning, transformation and exploratory analysis as opposed to machine learning development.

## Task 1 Problem Definition and Dataset Selection

### 1.1 Problem Definition and Motivation

The real-world problem that is being investigated in this project is predicting next day rainfall using historical Australian weather observation. Rainfall prediction can be important for daily planning and decision making as well as transport/logistics, operators, weather or environmental analysts, and agriculture driven careers. The dataset contains measurable variables such as temperature, humidity, pressure, rainfall, wind, cloud cover, and sunshine as well as the RainTomorrow outcome, the measurable predictor values and known outcome make the dataset makes this problem suitable for a data driven analysis. In regards to the dataset the historical weather observation proves useful because they provide past examples where weather conditions and the real next day outcome are both known, meaning that later this information can support supervised predictive modelling.

## 1.2 Dataset Selection and Relevance

The WeatherAUS dataset was selected because it contained real Australian weather observations and had a clear target variable, RainTomorrow. Therefore, the target variable could easily be used for a classification problem. [Click or tap here to enter text.](#) The dataset contains about 145,460 records and 23 original features which fulfils the selection criteria.

```
Records 145460
Features 23
<class 'pandas.DataFrame'>
RangeIndex: 145460 entries, 0 to 145459
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                  145460 non-null  str
1   Location              145460 non-null  str
2   MinTemp               143975 non-null  float64
3   MaxTemp               144199 non-null  float64
4   Rainfall              142199 non-null  float64
5   Evaporation           82670 non-null   float64
6   Sunshine              75625 non-null   float64
7   WindGustDir           135134 non-null  str
8   WindGustSpeed         135197 non-null  float64
9   WindDir9am            134894 non-null  str
10  WindDir3pm            141232 non-null  str
11  WindSpeed9am          143693 non-null  float64
12  WindSpeed3pm          142398 non-null  float64
13  Humidity9am           142806 non-null  float64
14  Humidity3pm           140953 non-null  float64
15  Pressure9am           130395 non-null  float64
16  Pressure3pm           130432 non-null  float64
17  Cloud9am              89572 non-null   float64
18  Cloud3pm              86102 non-null   float64
19  Temp9am               143693 non-null  float64
20  Temp3pm               141851 non-null  float64
21  RainToday             142199 non-null  str
22  RainTomorrow          142193 non-null  str
dtypes: float64(16), str(7)
memory usage: 25.5 MB
```

Fig 1

In terms of classification, since the target variable RainTomorrow has only two possible outcomes, Yes or No it can be classified as a binary classification problem. It is also large enough for a meaningful analysis since the data contains 10 years of daily weather observations across many locations in Australia. The data is made up of variables such as temperature, rainfall, evaporation, sunshine, wind direction and speed, humidity, pressure, cloud cover, location, and rainfall outcomes. RainTomorrow is the main target value, the Kaggle description states that it records whether the rain occurred the next day as yes or no. It also explains that yes was assigned when precipitation exceeded 1 mm in the 24 hours to 9 am. The dataset was created by the Australia Bureau of Meteorology making it reliable and accurate. It was compiled in the rattle project by Graham Williams.

## Task 2. Data Acquisition and Inspection

### 2.1 Dataset Structure and Initial Inspection

Firstly, to start the process of inspecting the data the WeatherAUS dataset was loaded to Python using pandas. As mentioned before the original dataset initially consisted of 145,460 observations and 23 variables. On the other hand, the variable captures several types of weather information such as temperature, rainfall, evaporation, sunshine, wind direction and speed etc. It also includes contextual variables such as the location, and date. Then using `select_dtypes()` the numerical and categorical column were identified in the dataset.

```
numeric_cols = df_raw.select_dtypes(include="number").columns
categorical_cols = df_raw.select_dtypes(include="str").columns

print("Numerical columns:")
print(numeric_cols.tolist())

print("\nCategorical columns:")
print(categorical_cols.tolist())
```

Output:

Numerical columns:

```
['MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation', 'Sunshine',
'WindGustSpeed', 'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am',
'Humidity3pm', 'Pressure9am', 'Pressure3pm', 'Cloud9am', 'Cloud3pm',
'Temp9am', 'Temp3pm']
```

Categorical columns:

```
['Date', 'Location', 'WindGustDir', 'WindDir9am', 'WindDir3pm',
'RainToday', 'RainTomorrow']
```

Fig 2

The identified target variable is `RainTomorrow`, and the `RainToday` variable is an input variable. To technically inspect the initial dataset structure, code snippets such as `df.head()`, `df_raw.info()`, `df_raw.shape`, `select_dtypes()`, `describe()` was used.

|                                 |               |               |               |              |              |               |               |               |      |
|---------------------------------|---------------|---------------|---------------|--------------|--------------|---------------|---------------|---------------|------|
| df_raw[numeric_cols].describe() |               |               |               |              |              |               |               |               |      |
| ✓ 0.2s                          |               |               |               |              |              |               |               |               |      |
|                                 | MinTemp       | MaxTemp       | Rainfall      | Evaporation  | Sunshine     | WindGustSpeed | WindSpeed9am  | WindSpeed3pm  | Hu   |
| count                           | 143975.000000 | 144199.000000 | 142199.000000 | 82670.000000 | 75625.000000 | 135197.000000 | 143693.000000 | 142398.000000 | 1428 |
| mean                            | 12.194034     | 23.221348     | 2.360918      | 5.468232     | 7.611178     | 40.035230     | 14.043426     | 18.662657     |      |
| std                             | 6.398495      | 7.119049      | 8.478060      | 4.193704     | 3.785483     | 13.607062     | 8.915375      | 8.809800      |      |
| min                             | -8.500000     | -4.800000     | 0.000000      | 0.000000     | 0.000000     | 6.000000      | 0.000000      | 0.000000      |      |
| 25%                             | 7.600000      | 17.900000     | 0.000000      | 2.600000     | 4.800000     | 31.000000     | 7.000000      | 13.000000     |      |
| 50%                             | 12.000000     | 22.600000     | 0.000000      | 4.800000     | 8.400000     | 39.000000     | 13.000000     | 19.000000     |      |
| 75%                             | 16.900000     | 28.200000     | 0.800000      | 7.400000     | 10.600000    | 48.000000     | 19.000000     | 24.000000     |      |
| max                             | 33.900000     | 48.100000     | 371.000000    | 145.000000   | 14.500000    | 135.000000    | 130.000000    | 87.000000     | 1    |

Fig 3

## 2.2 Initial Data Quality Findings

The main issue identified in the raw dataset was that many variables contained a lot of missing values, with some columns having much higher missingness than others. The missing values were not evenly distributed across all locations. Sunshine, Evaporation, Cloud3pm, and Cloud9am had much higher missingness than many other variables. Some locations also had at least one feature with 100% missing values. This showed that the missing-data problem was partly **location-dependent**.

```
missing_values = df_raw.isnull().sum()

missing_values          =          missing_values[missing_values          >
0].sort_values(ascending=False)

print("Missing values:")
print("\n" + missing_values.to_string())

print("Missing values percentage:")
missing_percentage = (missing_values / len(df_raw)) * 100
print("\n" + missing_percentage.to_string())

Output:
Missing values:

Sunshine          69835
Evaporation        62790
Cloud3pm           59358
Cloud9am           55888
Pressure9am        15065
Pressure3pm        15028
WindDir9am         10566
WindGustDir        10326
WindGustSpeed      10263
...
```

The dataset was also examined for duplicate records so later analysis would not be distorted, and zero duplicated rows were found. The distribution of the target variable was then checked to identify missing target values and whether the “Yes” and “No” outcomes were balanced. After removing missing target records, there were around 110k “No” values and 32k “Yes” values, showing that the target was imbalanced towards “No”.

Categorical values were inspected to check for unexpected labels, inconsistent spelling, or whitespace. Unique values such as Location, WindGustDir, WindDir9am, WindDir3pm, RainToday, and RainTomorrow were inspected, and whitespace was stripped as a formatting precaution. Major spelling or category errors were not found.

Numerical values were also checked for values outside the expected ranges. Cloud-cover values above the expected maximum of 8 were identified. Lastly, potential outliers were inspected so decisions could later be made about whether they should be modified or retained.

## Task 3. Data Cleansing and Transformation

### 3.1 Missing Values and Data Cleaning

We started the cleaning process by creating a deep copy of the raw dataset so that we could work on a clean copy while having a copy for comparison. Missing values such as sunshine, evaporation were identified and they were replaced by using the median of each individual column. This was done because the median is less affected by extreme values and skewed distribution in comparison to the mean. It was useful since some weather variable like rainfall and evaporation had clearly skewed and extreme values.

```
#reloading the original dataset starting fresh
```

```
df_clean = df_raw.copy(deep=True)
```

```
print("original Set:", df_clean.shape)
```

```
✓ 0.0s
```

Python

```
original Set: (145460, 23)
```

```
#filling each columns with its own mean or median value depending on the distribution of the data
```

```
numeric_columns = df_clean.select_dtypes(  
    include="number"  
) .columns
```

```
for col in numeric_columns:  
    df_clean[col] = df_clean[col].fillna(  
        df_clean[col].median()  
    )
```

```
✓ 0.0s
```

Python

```
#checking if any numeric values are still missing
```

```
print(  
    df_clean[numeric_columns]  
    .isna()  
    .sum()  
)
```

```
✓ 0.0s
```

Python

Categorical values could not be filled using a numerical median so the most common value, the mode was used to fill the missing categorical values. It was used on the WindGustDir, WindDir9am, WindDir3pm, and RainToday columns.

```
#filling in categorical columns missing value with the mode
```

```
categorical_fill_columns = [  
    "WindGustDir",  
    "WindDir9am",  
    "WindDir3pm",  
    "RainToday"  
]
```

```
for col in categorical_fill_columns:  
    df_clean[col] = df_clean[col].fillna(  
        df_clean[col].mode()[0]  
    )
```

```
✓ 0.0s
```

Python

```
#printing them to check there are no missing values left
```

```
print(  
    df_clean[categorical_fill_columns]  
    .isna()  
    .sum()  
)
```

```
✓ 0.0s
```

Python

```
WindGustDir    0  
WindDir9am     0  
WindDir3pm     0  
RainToday      0  
dtype: int64
```

RainTomorrow was handled differently because it is the target variable so if it is missing no known correct class would exist for that row. Instead of imputing RainTomorrow, records where the target was missing were dropped. Since dropping rows leaves gaps in the dataframe index, the index was then reset, and the total remaining missing values were checked.

Previously in Task 2 some location with very high missingness were identified, though these were investigated they were not permanently removed. Instead, the effect of removing locations where any feature exceeded 70% missingness was tested, but then it was reset back to the full raw dataset before the real cleaning process, so the location was not permanently removed. This was done after realising that removing the rows did not change the data much. After each imputation such as numerical imputation, categorical imputation, and dropping the missing target the missing value counts were printed, they were also printed again later as part of final validation.

## 3.2 Data Transformation and Encoding

The data transformation process was then started by checking the invalid values. The humidity values outside of 0 and 100, and the negative values in rainfall were checked. The cloud feature was checked to be above 0 and below 8 because cloud coverage is measured in oktas where 8 okta is full coverage.

```
print(
    col,
    "invalid values:",
    invalid_cloud.shape[0]
)
print(invalid_cloud[col])

df_clean.loc[
    df_clean["Cloud9am"] > 8,
    "Cloud9am"
] = 8

df_clean.loc[
    df_clean["Cloud3pm"] > 8,
    "Cloud3pm"
] = 8

print("Invalid cloud values after correction:")
invalid_cloud_after_correction = df_clean[
    (df_clean["Cloud9am"] < 0)
    | (df_clean["Cloud9am"] > 8)
    | (df_clean["Cloud3pm"] < 0)
    | (df_clean["Cloud3pm"] > 8)
]
print(invalid_cloud_after_correction.shape[0])
```

0.0s Python

Cloud9am invalid values: 0  
Series([], Name: Cloud9am, dtype: float64)  
Cloud3pm invalid values: 0  
Series([], Name: Cloud3pm, dtype: float64)  
Invalid cloud values after correction:  
0

The next step in transforming the data was to convert the categorical features to numerical values which will then enable the model to utilise them. This was done with the help of encoding namely One-Hot Encoding and Sine-Cosine Encoding for the directions as they are circular. Sine Cosine Encoding helped reduce the number of features in the dataset after cleaning.

```
wind_angles = {
    "N": 0,
    "NNE": 22.5,
    "NE": 45,
    "ENE": 67.5,
    "E": 90,
    "ESE": 112.5,
    "SE": 135,
    "SSE": 157.5,
    "S": 180,
    "SSW": 202.5,
    "SW": 225,
    "WSW": 247.5,
```



```

        "W": 270,
        "WNW": 292.5,
        "NW": 315,
        "NNW": 337.5
    }
    wind_cols = [
        "WindGustDir",
        "WindDir9am",
        "WindDir3pm"
    ]
    for col in wind_cols:
        angle = df_clean[col].map(wind_angles)

        df_clean[col + "_sin"] = np.sin(
            np.radians(angle)
        )

        df_clean[col + "_cos"] = np.cos(
            np.radians(angle)
        )
    df_clean = df_clean.drop(columns=wind_cols)

```

The total number of features in the cleaned dataset did however go up to 76 mainly because of one-hot encoding the location feature. This step is vital for the model to be able to location as it is an important feature.

```

# One-hot encoding Location
df_clean = pd.get_dummies(
    df_clean,
    columns=["Location"],
    dtype=int
)

# Checking whether any categorical columns remain
categorical_cols = df_clean.select_dtypes(
    include=["object", "category"]
).columns.tolist()

print("Remaining categorical columns:")
print(categorical_cols)

print("\nFinal dataset shape:")
print(df_clean.shape)

print("\nFinal dataset preview:")
print(df_clean.head())

```

Remaining categorical columns:  
[]

Final dataset shape:  
(142193, 76)

|   | Location_Townsville | Location_Tuggeranong | Location_Uluru \ |
|---|---------------------|----------------------|------------------|
| 0 | 0                   | 0                    | 0                |
| 1 | 0                   | 0                    | 0                |
| 2 | 0                   | 0                    | 0                |
| 3 | 0                   | 0                    | 0                |
| 4 | 0                   | 0                    | 0                |

|   | Location_WaggaWagga | Location_Walpole | Location_Watsonia \ |
|---|---------------------|------------------|---------------------|
| 0 | 0                   | 0                | 0                   |
| 1 | 0                   | 0                | 0                   |
| 2 | 0                   | 0                | 0                   |
| 3 | 0                   | 0                | 0                   |
| 4 | 0                   | 0                | 0                   |

|   | Location_Williamtown | Location_Witchcliffe | Location_Wollongong \ |
|---|----------------------|----------------------|-----------------------|
| 0 | 0                    | 0                    | 0                     |
| 1 | 0                    | 0                    | 0                     |
| 2 | 0                    | 0                    | 0                     |
| 3 | 0                    | 0                    | 0                     |
| 4 | 0                    | 0                    | 0                     |

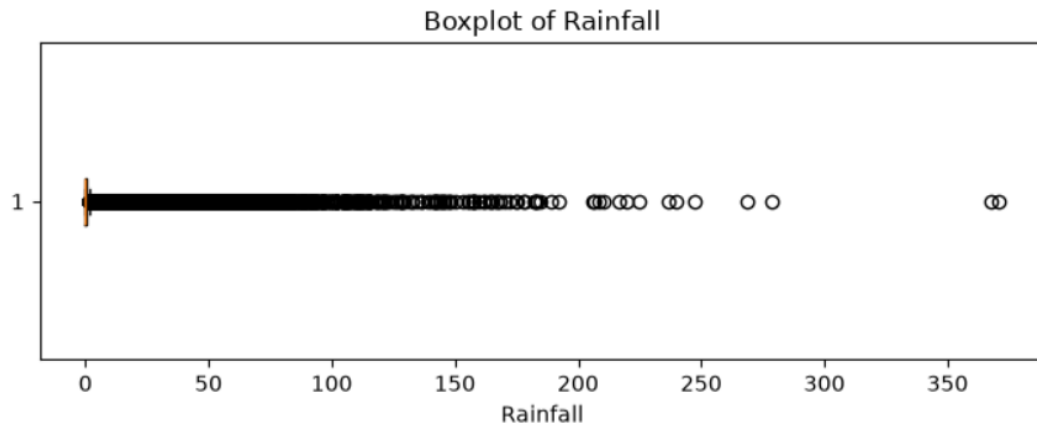
  

|   | Location_Woomera |
|---|------------------|
| 0 | 0                |
| 1 | 0                |
| 2 | 0                |
| 3 | 0                |
| 4 | 0                |

[5 rows x 76 columns]

### 3.3 Outlier Investigation and Final Validation

Then, the outliers were looked at with the help of box plots and coding in some cases. However, it was hard to decide what values can be left out of the dataset because weather is unpredictable and does not follow a set pattern resulting in extreme values.



For example, even though it looks like rainfall has 2 outliers as seen in the box plot above. They can be a result of rare extreme weather conditions and are thus valuable to the future model.

However, one outlier was found in the Evaporation feature as seen below.

|        | Date       | Location      | Evaporation | MaxTemp | Sunshine | \ |
|--------|------------|---------------|-------------|---------|----------|---|
| 42358  | 2016-12-19 | Williamtown   | 145.0       | 23.9    | NaN      |   |
| 8831   | 2016-11-11 | Cobar         | 86.2        | 35.3    | NaN      |   |
| 106968 | 2013-01-07 | Woomera       | 82.4        | 45.3    | NaN      |   |
| 105935 | 2010-01-09 | Woomera       | 81.2        | 41.7    | 13.2     |   |
| 107290 | 2013-12-23 | Woomera       | 77.3        | 27.2    | NaN      |   |
| 23954  | 2017-01-03 | NorfolkIsland | 74.8        | 26.1    | NaN      |   |
| 105965 | 2010-02-08 | Woomera       | 72.2        | 39.7    | 13.0     |   |
| 107968 | 2015-11-01 | Woomera       | 70.4        | 34.2    | NaN      |   |
| 105949 | 2010-01-23 | Woomera       | 70.0        | 33.1    | 11.7     |   |
| 107668 | 2015-01-05 | Woomera       | 68.8        | 38.3    | NaN      |   |
| 107599 | 2014-10-28 | Woomera       | 68.8        | 26.7    | NaN      |   |
| 108390 | 2016-12-27 | Woomera       | 67.2        | 31.5    | NaN      |   |

The Williamtown record was then further investigated, and it was found that the value was very different from the others in the same 2 weeks.

```
#checking williamtown around the same date as the outlier in evaporation to see
if there are any other outliers in the other columns
df_raw.loc[
    (df_raw["Location"] == "Williamtown") &
    (df_raw["Date"] >= "2016-12-10") &
    (df_raw["Date"] <= "2016-12-28"),
    [
        "Date",
        "Location",
        "Evaporation",
        "MaxTemp",
        "Rainfall",
```

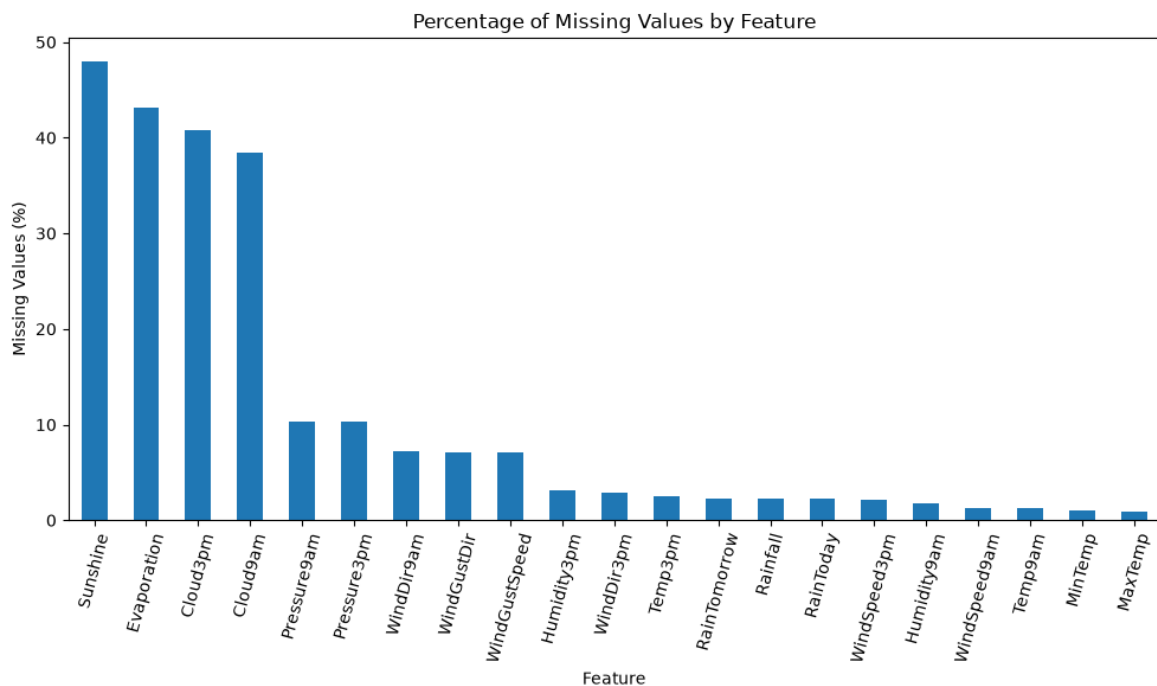
|               |            |             |       |      |      |      |
|---------------|------------|-------------|-------|------|------|------|
| "Humidity3pm" |            |             |       |      |      |      |
| ]             |            |             |       |      |      |      |
| ]             |            |             |       |      |      |      |
| 42353         | 2016-12-14 | Williamtown | NaN   | 38.2 | NaN  | 17.0 |
| 42354         | 2016-12-15 | Williamtown | NaN   | 19.9 | NaN  | 95.0 |
| 42355         | 2016-12-16 | Williamtown | NaN   | 26.8 | NaN  | 74.0 |
| 42356         | 2016-12-17 | Williamtown | NaN   | 34.3 | NaN  | 33.0 |
| 42357         | 2016-12-18 | Williamtown | NaN   | 23.4 | NaN  | 58.0 |
| 42358         | 2016-12-19 | Williamtown | 145.0 | 23.9 | 58.2 | 52.0 |
| 42359         | 2016-12-20 | Williamtown | 8.6   | 35.1 | 0.0  | 37.0 |
| 42360         | 2016-12-21 | Williamtown | 11.0  | 25.9 | 0.0  | 65.0 |
| 42361         | 2016-12-22 | Williamtown | 7.4   | 24.5 | 0.0  | 58.0 |
| 42362         | 2016-12-23 | Williamtown | 6.2   | 26.7 | 0.0  | 55.0 |
| 42363         | 2016-12-24 | Williamtown | NaN   | 32.2 | NaN  | 44.0 |

Thus, this record was replaced with the median evaporation as it was suspected of being an incorrect measurement. All other visible outliers were allowed as they were considered extreme but plausible values.

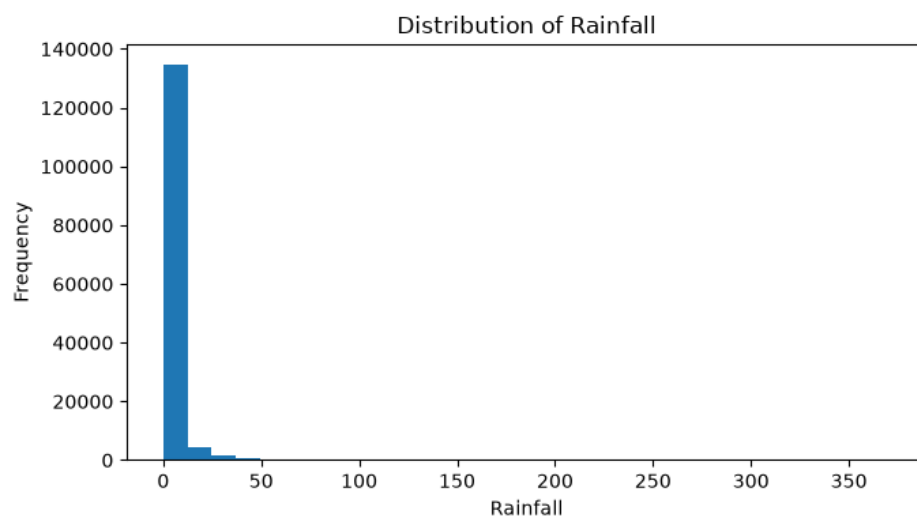
## Task 4. Exploratory Data Analysis and Visualisation

### 4.1 Descriptive Analysis and Data Distributions

In the missing value graph, it could be seen that the missingness was not evenly distributed across the dataset, with Sunshine, Evaporation, Cloud3pm, and Cloud9am having the highest percentages. This tells us that the missing data problem was concentrated weather measurements as opposed to being equally distributed across all features.

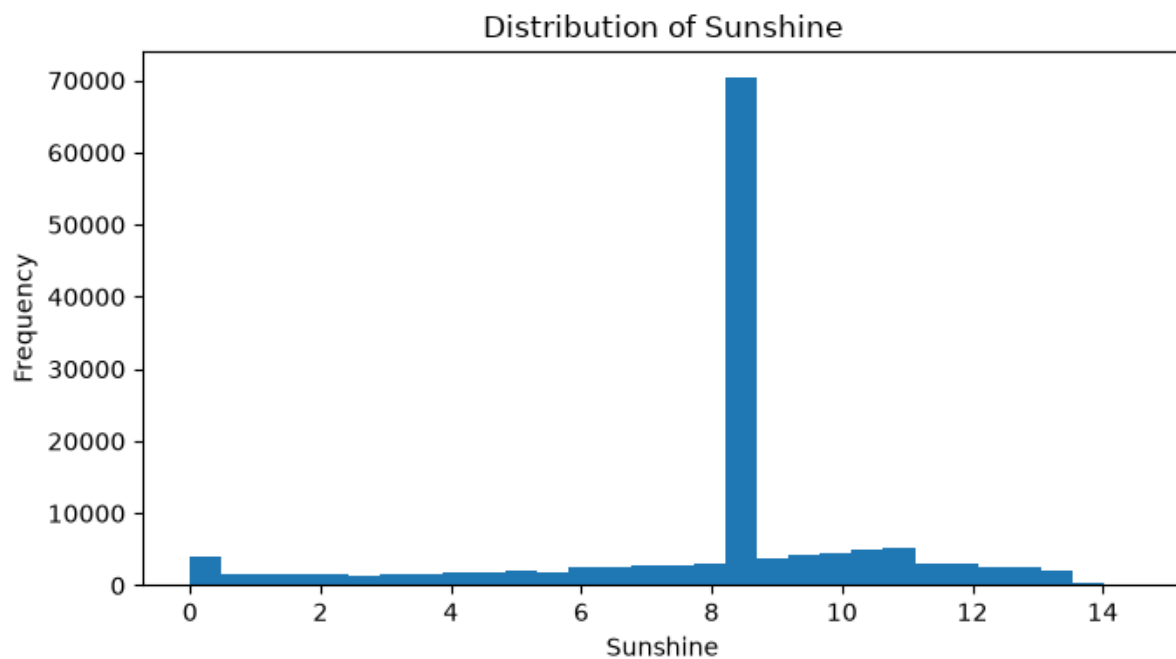


In terms of temperature distributions, the MinTemp and MaxTemp distributions were continuous and approximately centred around their middle temperature ranges, with fewer observations found at the extremes. Compared to variables such as rainfall, the temperature variables showed much less skewed distributions. The rainfall distribution was strongly right skewed, with most of the findings close to zero and a small number extending to very high rainfall values. This indicates that low or no rainfall took place much more frequently than extremely heavy rainfall events did.



WindGustSpeed also showed a positively skewed distribution, although the skew was less extreme compared to rainfall. Most of the wind gust observations were concentrated around the lower to middle speed range, while very strong wind gusts were comparatively less common. In Humidity3pm there was a broader distribution across the 0–100% range, with most observations being concentrated around the middle of the range.

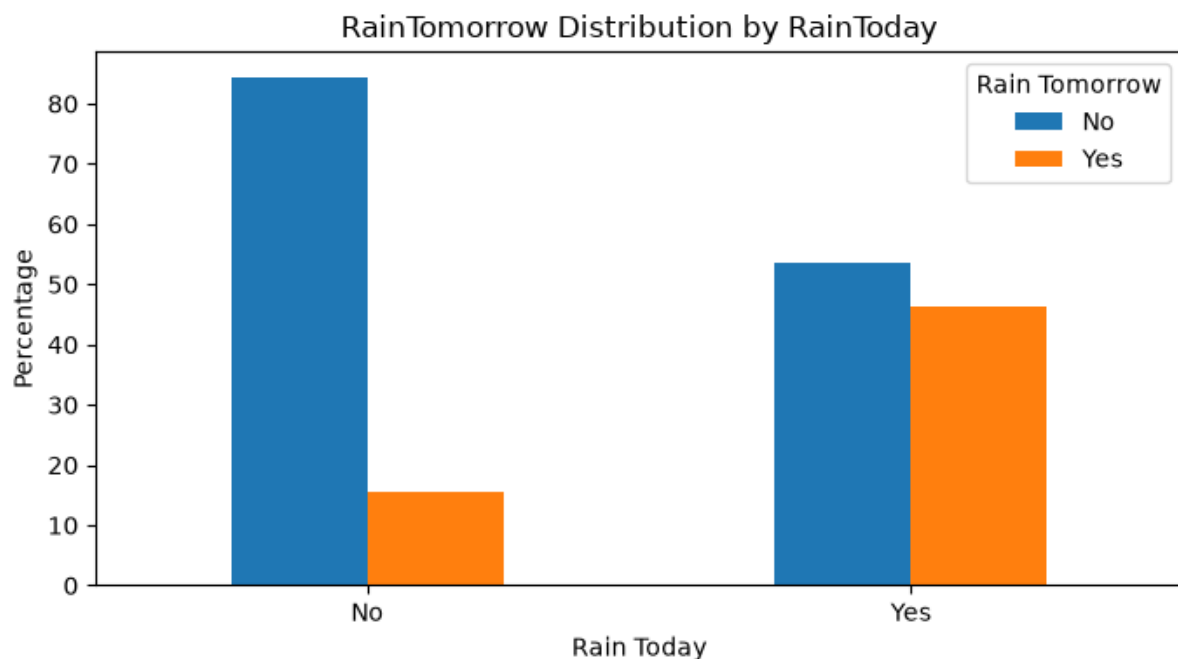
Due to the imputation effect in Sunshine and Cloud3pm, a noticeable spike can be seen in the Sunshine distribution around the median value used during imputation. Meanwhile, a similar concentration can also be found in Cloud3pm, which can be explained by many missing observations being replaced with the same median value. This demonstrates that although median imputation removed the missing values, it also changed the original distribution by introducing many identical values.



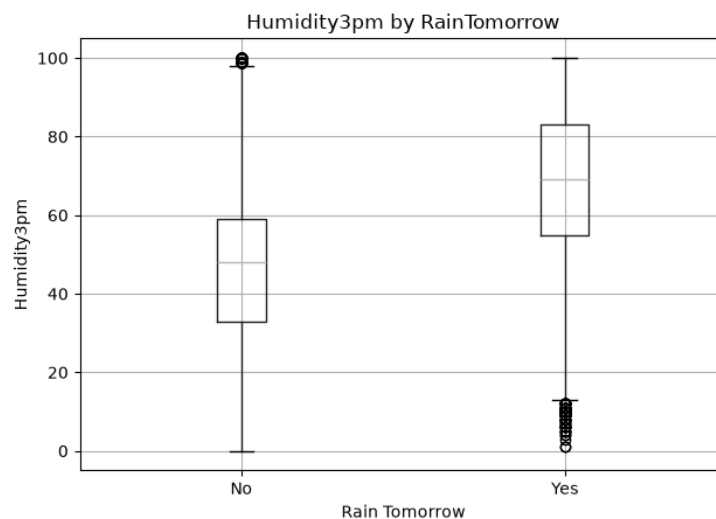
## 4.2 Target Distribution and Variable Relationships

The distribution of RainTomorrow showed a clear class imbalance, with around 110k “No” observations compared with roughly 32k “Yes” observations. This shows us that the dataset contains substantially more non-rainy next-day outcomes compared to rainy outcomes.

Between RainToday and RainTomorrow, a noticeable relationship was identified. When RainToday was “No”, only 15.6% of the observations resulted in rain the following day. Meanwhile, when RainToday was “Yes”, the proportion of RainTomorrow also equalling “Yes” increased to approximately 46.4%. This suggests that rainfall on the current day is associated with a greater occurrence of rainfall on the following day.



Humidity3pm showed one of the clearest differences between the RainTomorrow groups. In observations where RainTomorrow was “Yes”, there was typically higher afternoon humidity compared to observations where RainTomorrow was “No”. This means that the mean Humidity3pm increased from approximately 46.64% for “No” to 68.29% for “Yes”.



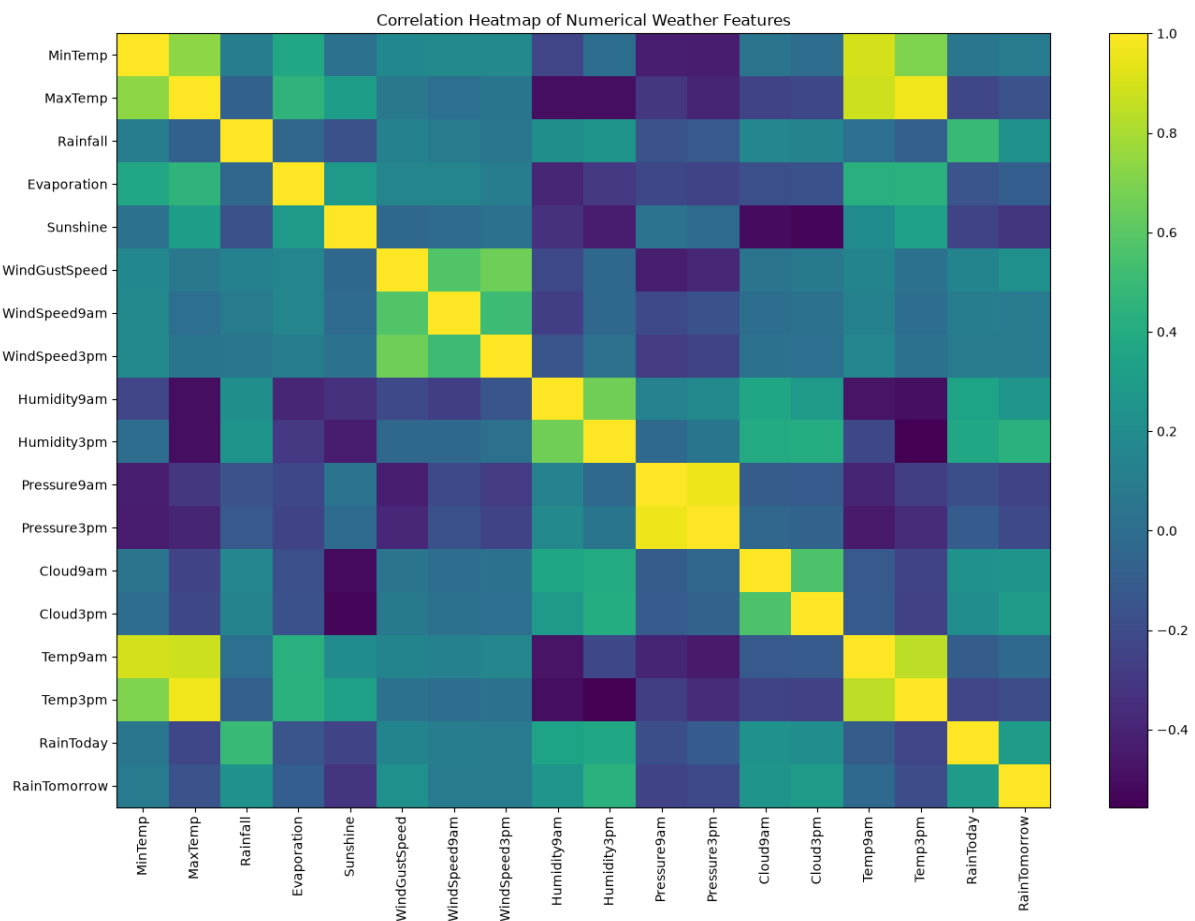
Sunshine demonstrated the opposite pattern. If RainTomorrow was equal to “Yes”, it was associated with lower sunshine values. The mean sunshine value decreased from approximately 8.48 hours for “No” to 6.33 hours for “Yes”.

Pressure3pm also showed differences between the target classes, although the separation was much smaller compared to Humidity3pm. The “Yes” group had a slightly lower average atmospheric pressure of about 1012.58 compared with 1016.02 for “No”.

WindGustSpeed was shown to be higher when RainTomorrow was “Yes”, with averages around 45.41 km/h compared to 38.33 km/h for “No”. However, the sizable overlap between the two groups suggests that wind gust speed alone cannot clearly separate the two outcomes.

The percentage of RainTomorrow equalling “Yes” varied significantly between locations. An example of this was that locations such as Portland and Walpole showed some of the highest proportions, while locations like Woomera, Uluru and Alice Springs were among the lowest percentages. This tells us that geographic location is associated with substantial differences in next-day rainfall frequency. This variation supports retaining Location as a useful predictor rather than removing it from the dataset.

## 4.3 Correlation and Key EDA Findings



Looking at the correlation heat map above we can notice the yellow diagonal line and some other spots with high correlation, but the data is mostly weakly correlated. This was particularly useful during cleaning because we did not have to delete any features. MaxTemp and Temp3pm have a high correlation because they are temperature values from the same day, however, we decided to keep Temp3pm because weather can be unpredictable and Temp9am also had a higher correlation with MaxTemp which implies that the max temperature could have been reached at any point of time in those 6 hours.

```
# Showing which numerical features have the strongest
# relationship with the RainTomorrow target
target_correlations = (
    correlation_matrix["RainTomorrow"]
    .sort_values(ascending=False)
)

print("Correlations with RainTomorrow:")
print(target_correlations.round(3))
```

Output:

Correlations with RainTomorrow:

|              |       |
|--------------|-------|
| RainTomorrow | 1.000 |
| Humidity3pm  | 0.440 |

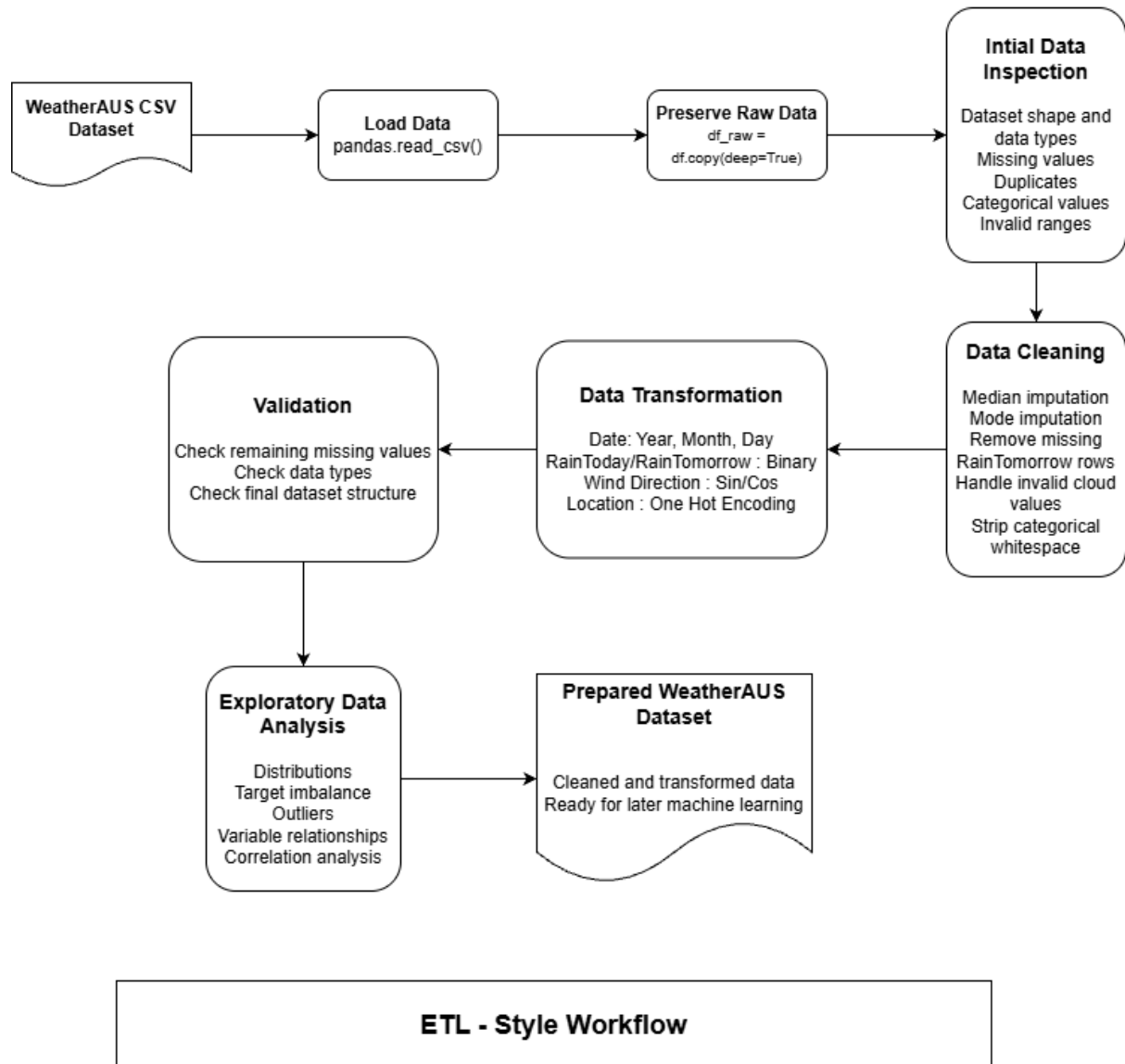


```
RainToday      0.307
Cloud3pm       0.295
Humidity9am    0.255
Cloud9am       0.249
Rainfall       0.235
WindGustSpeed  0.225
WindSpeed9am   0.090
WindSpeed3pm   0.087
MinTemp        0.084
Temp9am        -0.026
Evaporation    -0.091
MaxTemp        -0.159
Temp3pm        -0.190
Pressure3pm    -0.215
Pressure9am    -0.234
Sunshine       -0.324
Name: RainTomorrow, dtype: float64
```

This code snippet proves what was discussed earlier: RainTomorrow mostly depends on Humidity3pm and RainToday which aligns with real world observations.

## 5. Data Pipeline and Reproducibility

### WeatherAUS Data Pipeline Diagram



## 6. Progress Summary

The phase 1 of this project focused on taking the original WeatherAUS dataset through acquisition, inspection, cleaning, transformation and validation to produce a cleaned dataset that can be used for later machine learning work. Some of the biggest data quality issues found were missing values in variables mentioned in the above sections. There were no duplicate records, but there were missing target values, invalid cloud cover values and numerical outliers. These problems were handled using methods such as median imputation, mode imputation, removal of rows with missing target values, handling invalid cloud values, stripping whitespace from categorical values and investigating outliers rather than removing them automatically. Using these methods, we were able to convert the data into a numerical, model-compatible format while preserving important information. Overall, Phase 1 produced a cleaned, transformed and validated dataset of approximately 142,193 records that is ready for later machine-learning development. However, target imbalance, imputed values and retained statistical outliers should still be considered during the modelling stage.